Trusting Trust, Revisited

A Complete Project Companion — Everything Built, Why It Works, and How to Explain It

This document exists for one purpose: so that you can sit in front of your faculty and explain every part of this project confidently, in your own words, including the parts that are not yours (Thompson's attack, Wheeler's defense) and the parts that are (the extensions, the arms race, the original detection method, and its proof). It does not just list what was built — it explains why each piece exists, how it actually works, and what to say if you are asked to defend any specific claim.

Read it once start to finish before your presentation. Then use the Anticipated Questions section (Part VII) and the Glossary (Part VIII) as last-minute references.

Scope	Phases 1 through 6 (Parts 1–4), complete
Core languages/tools	C++17, Python 3 (tooling), bash, git
Final novelty position	9 / 10, argued and defended in Part V
Document purpose	Personal mastery + faculty explanation

Contents

Part I — The Core Idea, In Plain Language
Part II — The Project at a Glance
Part III — Phase-by-Phase Walkthrough
  3.1 Phase 1: The Honest Compiler
  3.2 Phase 2: Teaching It To Lie
  3.3 Phase 3: The Quine
  3.4 Phase 4: Self-Replication
  3.5 Phase 5: The Defense (DDC)
  3.6 Phase 6, Part 1: Beyond Hashing
  3.7 Phase 6, Part 2: The Arms Race
  3.8 Phase 6, Part 3: The Original Method
  3.9 Phase 6, Part 4: The Formal Proof
Part IV — Literature Review Summary (24 Sources)
Part V — The Novelty Argument
Part VI — Advantages and Disadvantages
Part VII — Anticipated Questions From Faculty
Part VIII — Glossary of Key Terms
References

Part I — The Core Idea, In Plain Language

Start here, because everything else in this document is detail on top of one idea. When you compile a program, you trust that the compiler faithfully turns your source code into a running program — nothing more, nothing less. Source code review only works if that trust is justified.

In 1984, Ken Thompson showed it does not have to be. A compiler can be modified to recognize one specific program it is compiling (say, a login check) and quietly insert extra logic — a backdoor — into the output, even though that logic appears nowhere in the program's source. Worse, the compiler can also recognize when it is compiling **itself**, and reinsert this same trick into the next version of the compiler it produces. Once this happens, you can delete the malicious code from every source file you own, recompile with the infected tool, and the backdoor comes back — because the infected binary, not any file you can read, is what's carrying it forward.

The defense most widely accepted today is David Wheeler's **Diverse Double Compiling (DDC)**, from 2005. The idea: build the compiler's own source code using two independent compilers — one you trust, one you don't — then use *both* resulting compilers to build the source again. If both original compilers were honest, the two final results must be byte-for-byte identical. You never have to read or trust the suspect compiler's source at all — you only compare outputs.

> **The one-sentence version:** this project builds Thompson's attack for real, builds Wheeler's defense for real, then deliberately breaks that defense, fixes it, and adds an original detection method with a mathematical proof behind it — all in a small, safety-scoped C++ codebase you can read start to finish in an afternoon.

Part II — The Project at a Glance

Six phases, numbered 1 through 6, with Phase 6 split into four parts because that is where the project's own original contribution lives. Read this table first; Part III explains each row in depth.

Phase	What it is	The one-line result
1	A real TinyLang → C++ compiler, built from scratch	9/9 tests pass; 3 runs hash byte-identical
2	A harmless marker injected into one function's output	Confirmed scoped to its target only
3	A C++ quine (a program that prints its own source)	Verified stable across 2 generations
4	Injection + quine fused into genuine self-replication	Infection survives deletion; stable by gen. 3
5	A working Diverse Double Compiling (DDC) harness	Real alert on infected code; real pass on clean
6.1	Independent detectors beyond hashing (static + dynamic)	Static scanner and fuzzer built and tested
6.2	A real two-round attacker/defender arms race	Evasion fools DDC once, then DDC is fixed
6.3	An original detection method (the project's own idea)	Metamorphic testing applied to this problem

6.4	A formal soundness proof for that method	Proof's prediction confirmed by rerunning the tool

Part III — Phase-by-Phase Walkthrough

Each phase below follows the same structure: what it is, why it exists, how it actually works, and what was verified — not just claimed. This is the part to re-read until you can explain each phase without looking at notes.

3.1 — Phase 1: The Honest Compiler

Before you can demonstrate a compiler lying, you need a compiler that tells the truth. Phase 1 is a complete, working transpiler for a small language designed for this project, called **TinyLang**, that compiles down to C++. It has three real parts: a lexer that turns source text into tokens using a deterministic finite automaton (DFA), a recursive-descent parser that builds an explicit abstract syntax tree (AST) from those tokens, and a code generator that walks the AST and emits C++.

Why determinism matters here: every later phase that compares two builds (Phase 5 onward) only works if compiling the same input twice always produces the exact same output. Phase 1's code generator was deliberately written to avoid anything that could break this — no hash-map iteration order leaking into output order, no timestamps, nothing non-reproducible.

What was verified: 9 out of 9 functional tests pass (covering authentication logic, variable reassignment, nested conditionals, multiple functions). The same input file was compiled three separate times and all three outputs hashed identically with SHA-256 — proving the determinism claim rather than assuming it.

3.2 — Phase 2: Teaching It To Lie

This is Thompson's “Rule 1” in miniature: the compiler is modified to recognize one specific function by name — `check_login` — and, only when compiling that function, silently insert an extra line of code into the output before the function's real body. The injected code is deliberately inert: a comment labeled `[MARKER]` plus a harmless line that prints to `stderr`. It never touches the function's actual return value.

Why inert, not a real backdoor: the entire mechanism Thompson described — detection, injection, later self-replication — is fully demonstrated whether the payload is a real credential bypass or a harmless printed marker. Making it harmless preserves every structural lesson of the project while keeping the artifact safe to build, run, and submit.

What was verified: the marker appears in generated code for `check_login` and nowhere else; a second, unrelated function compiled through the same modified compiler shows no trace of it. The real authentication logic (does the password actually match?) was tested and confirmed unchanged.

3.3 — Phase 3: The Quine

A **quine** is a program whose output, when run, is an exact copy of its own source code. This is the mechanical building block Phase 4 needs: for the infection to survive being deleted from source, the compiler has to be able to reproduce its own malicious logic as part of its normal output whenever it compiles itself.

Why this is hard in C++ specifically: a quine's source string has to somehow contain a copy of itself, including the exact syntax that closes that same string. C++11 raw string literals (`R"(...)"`) make most of

this easy by not requiring escaped quotes or newlines — but the one part they can't avoid is a literal copy of their own closing delimiter appearing inside their own content, which would make the compiler think the string ends early, corrupting everything after it.

The fix used: the dangerous closing-quote character is represented with a two-character placeholder (%c) instead of a literal quote while stored in source, and only reassembled into a real quote character at runtime via string substitution — so the source file itself never contains the self-terminating sequence, but the program's output does, correctly.

What was verified: the quine was compiled, run, and its output diffed byte-for-byte against its own source — identical. The output was then compiled and run *again* (a second generation) to confirm the property holds under repeated compilation, not just once.

3.4 — Phase 4: Self-Replication (the centerpiece)

This is Thompson's “Rule 2,” and the single most important demonstration in the project. Phase 2's injection and Phase 3's quine technique are fused: the compiler gains a **self-recognition hook** that checks whether the source it is currently compiling looks like its own source code (by searching for a signature unique to its own tokenizer, e.g. the exact text `std::vector<Token> tokenize` inside a file called `lexer.hpp`). If the signature matches, the compiler regenerates *new* versions of its own source files — carrying both the Phase 2 marker-injection logic and this very self-recognition logic forward — using payloads embedded inside the running binary, regardless of what the on-disk source actually contains.

> **The actual demonstration, step by step:**
> 1. A clean copy of the Phase 1 compiler's source is committed to git.
> 2. Confirmed by direct text search: zero occurrences of the word “MARKER” anywhere in that committed source.
> 3. The already-built infected compiler is pointed at that clean, committed source with a `--replicate` command.
> 4. The regenerated files it produces **do** contain the marker injection logic — despite the source it just read containing none of it.
> 5. The newly regenerated compiler is built and run: it still injects the marker on normal input, and its real authentication logic is still unaffected.
> 6. This is repeated for a third generation and found to produce byte-identical output to the second — a **stable fixed point**, meaning the infection will propagate forever under this process, exactly as Thompson described.

A real bug found during development: an early version of the self-replication logic substituted two placeholder tokens in the wrong order, which caused one placeholder's replacement to insert a second, nested copy of the other placeholder earlier in the text than intended — breaking the regenerated file's syntax. This was only caught by actually compiling the regenerated output and watching it fail, then fixed by reordering the substitutions. This is worth mentioning to faculty: it is direct evidence the demonstration is real and was actually run, not just described.

3.5 — Phase 5: The Defense (Diverse Double Compiling)

Phase 5 implements Wheeler's DDC exactly as described in Part I, adapted for this project's transpiler-based toolchain: `compiler_A` is built directly from clean source using trusted, unmodified g++; `compiler_B` is built via the suspect compiler's own `--replicate` mode and then compiled with the same g++. Both resulting compilers are then used to compile the same representative TinyLang test file, and the two outputs are hashed and compared.

What was verified, both directions: run against the real infected compiler from Phase 4, the harness correctly reports `[ALERT] Discrepancy detected!` with exit code 1, and the diff output pinpoints the exact injected lines automatically — without ever being told what to look for. Run against two independently built *clean* compilers, it correctly reports `[SUCCESS]` with exit code 0. Both the positive and the negative case were confirmed, proving the harness discriminates rather than always alarming.

3.6 — Phase 6, Part 1: Beyond Hashing

Everything from here on is extension work added specifically to push the project's novelty beyond “a correct reproduction of known results.” Part 1 builds detectors that do not rely on comparing two full builds the way DDC does.

Static I/O scanner. TinyLang's grammar has no input/output primitive at all — no print statement, nothing. This means any I/O call (such as `std::cerr`) appearing in generated C++ is suspicious by construction, regardless of what text it prints. This check needs no known signature and no reference compiler.

Behavioral fuzzer. Compiles the generated `check_login` into a small test driver and runs it across many (user, password) input pairs, comparing observed behavior between a clean build and a candidate build. Run in two modes deliberately: checking only the return value misses the Phase 2 injection completely (0 divergences out of 49 inputs tested), because the injected marker is designed to be inert with respect to return value; checking return value *and* captured stderr output catches it on every single input (49 out of 49). This is a genuine, measured finding about test coverage, not a flaw in the fuzzer.

Multi-target injection. The hardcoded check for the single name `check_login` was generalized to a keyword match (any function name containing “login,” “auth,” “password,” or “credential”). Tested against a file with three matching functions and one unrelated decoy: all three matches were correctly injected, the decoy was correctly left untouched.

Quantification. Real measurements were taken rather than assumed: the injection logic adds 15–29 lines and 700–1400 bytes to the compiler's own source depending on variant; a full detection matrix across five detectors and four compiler variants was run and recorded; and timing was measured from N=10 to N=1000 synthetic functions, honestly noting that at this scale, fixed process-startup overhead dominates the true algorithmic cost — a limitation stated plainly rather than hidden.

3.7 — Phase 6, Part 2: The Arms Race

This is where the project stops being purely defensive and starts modeling an adaptive attacker — a real two-round contest between attack and defense, each side's claim backed by an actual run, not a thought experiment.

Round 1 — the evasive compiler. A new compiler variant computes a hash of the exact raw bytes of whatever source file it is asked to compile, and compares it against a hardcoded hash of this project's own published test file. If they match exactly, the marker injection is suppressed for that one compilation; any other input is infected normally. This mirrors a real, well-known pattern outside software too: detecting a standard test and behaving differently only during it (the same general idea as automotive emissions test-cycle detection). Result: the Phase 5 DDC harness, which always tested with that one published file, was genuinely fooled — it reported `[SUCCESS]` against a compiler that was, in fact, infected.

Round 2 — the countermeasure. Rather than trying to detect or reverse-engineer the evasion logic, the DDC harness was modified to generate a fresh, randomized TinyLang test file on every run instead of reusing the one published fixture. Since the evasive compiler's suppression is keyed to one exact, specific byte sequence, a freshly randomized file never matches it, and the injection fires normally again. Verified across five independent random seeds: detection restored every time. Verified against a genuinely clean pair across three more seeds: zero false positives.

The ensemble detector, and two real bugs. All checks built across the project were combined into one scored verdict (flag as infected if 2 or more of several independent checks agree). Building this surfaced two genuine implementation bugs, both found by attacking the ensemble with the project's own strongest evasion rather than by inspection:

- **Bug 1 — a shared blind spot.** The first version fed several of the supposedly independent detectors the same fixed, predictable input DDC uses — so they silently inherited the exact blind spot Round 1 exploited. Against the evasive compiler, the ensemble produced a false “CLEAN” verdict. Fixed by giving every detector, not just DDC, a freshly randomized input each run, and by removing any fixed-fixture pathway from the ensemble entirely rather than merely outvoting it.
- **Bug 2 — a crash misread as a detection.** An early version of the randomized test generator also randomized the target function's *name*, which silently broke the fuzzer's driver (it calls a fixed function name), causing a compilation failure whose exit code was wrongly interpreted as a genuine finding. Fixed by keeping the function name constant and randomizing only literal content, which is sufficient on its own to defeat the fixture-hash evasion.

After both fixes, the corrected 7-detector ensemble was re-verified across 5 compiler variants and 3 random seeds each (15 runs total): zero false positives on clean code, correct detection on every infected variant including the evasive one.

3.8 — Phase 6, Part 3: The Original Method

This is the project's own idea, and the strongest basis for its novelty claim. Every detector built so far needs something external: DDC needs a second, trusted compiler; the scanners need to know a suspicious category or pattern in advance. This method needs neither.

The core idea: apply the general software-testing technique called **metamorphic testing** — defining transformations a correct program must treat as meaningless, then checking whether a single, possibly-infected program is even consistent with itself across them — specifically to trusting-trust-style compiler backdoor detection. A literature search confirmed this specific combination had not been published before (metamorphic testing has been applied to ordinary compiler correctness bugs, and trusting-trust/DDC is its own established area, but not the two together for this purpose).

Two concrete relations, built from TinyLang's own grammar:

- **MR1 (comment insertion).** TinyLang's own lexer explicitly strips `//` comments before producing any tokens, so adding a comment anywhere in a source file is *guaranteed by the grammar itself* to produce an identical parse tree. A correct compiler's output must therefore be byte-identical with or without the comment — any difference is a violation.
- **MR2 (unrelated decoy insertion).** Appending an unrelated, independently-named function to a file should not change how an earlier, unrelated function in that same file is compiled, since a correct compiler processes each function on its own terms.

What was verified, with the result predicted before it was run: both relations stayed silent (no violation) on the clean compiler and on the plain, multitarget, and obfuscated infected variants — all of which only read a function's own name, a legitimate part of its syntax tree. Both relations *fired* on the evasive compiler from Round 1, because its evasion logic depends on the whole file's raw bytes, which both MR1 and MR2 deliberately perturb. This is architecturally different from every other detector built: it was run with a single compiler binary as its only argument, no reference compiler present at all, and still caught the infection.

3.9 — Phase 6, Part 4: The Formal Proof

Part 3 showed the method working empirically. Part 4 proves *why* it must work, using definitions grounded directly in this project's own real source code — not an abstract argument about compilers in general.

Definition 1 (AST-faithful / honest compiler). A compiler C is honest if its output depends only on the program's abstract syntax tree — formally, there exists some function G taking only an AST as input such that C(S) = G(AST(S)) for every source text S. Nothing about raw bytes, file names, or invocation history is allowed to influence the result.

Definition 2 (grammar-null transformation). A transformation T is grammar-null if AST(T(S)) equals AST(S) for every S — it changes the text but provably not the program's meaning. MR1's comment insertion is grammar-null by direct inspection of `lexer.cpp`'s own comment-handling code.

> **The theorem.** If C is AST-faithful and T is grammar-null, then C(T(S)) = C(S) for every S. *Proof:* C(T(S)) = G(AST(T(S))) = G(AST(S)) = C(S), by direct substitution using the two definitions above.
>
> **The useful direction (contrapositive).** If C(T(S)) does not equal C(S) for some grammar-null T, then C is *not* honest. This is not a heuristic — it is a direct logical consequence of the two definitions, and it is what the metamorphic tester is actually exploiting each time it reports a violation.

MR2 needs one more premise (Definition 3, compositional codegen): that a correct generator builds each top-level function's output from only that function's own subtree, never its siblings'. This is verified by reading the real Phase 1 `genFuncDef` function directly — it takes only its own node and a fresh, locally-seeded symbol table, with no access to sibling functions or global state.

The precise, honest limitation — and why it strengthens the argument rather than weakens it. The theorem proves a violation is sound evidence of dishonesty, but silence does *not* prove honesty. This can be stated exactly: the proof splits backdoor logic into two distinct classes. **Class A** (AST-external dependence, like the evasive compiler's whole-file hash check) is provably detectable by this method with no reference

compiler needed. **Class B** (AST-internal but illegitimate logic, like checking a function's own name) is, by the theorem itself, something no grammar-null transformation can ever expose — because such a transformation leaves the very AST fields Class B legitimately reads completely unchanged. This is exactly why the plain, multitarget, and obfuscated variants stayed undetected by MR1/MR2 in Part 3: not a weakness found by accident, but a predicted, provable boundary.

Why this means the ensemble is structurally necessary, not just convenient. Only a detector holding an actual reference compiler (DDC) is class-agnostic — it does not need to know which class of dishonesty it is looking at, only that the candidate disagrees with a trusted implementation. Every reference-free method is mathematically confined to one class. Combining detector classes is therefore required by the proof, not just good practice.

Part IV — Literature Review Summary (24 Sources)

Twenty-four real, citable sources were surveyed, spanning 1974 to 2026. Rather than just listing them (see References at the end of this document for the full list), here is what they collectively establish, grouped into five chronological clusters.

1974–2009 — Foundational attack & defense (4 sources)

Establishes the threat model (Karger & Schell's 1974 Multics review; Thompson's 1984 lecture) and the one broadly accepted countermeasure, DDC (Wheeler, 2005 and 2009). Together they prove the attack and its defense are both real and formally sound — but neither ships as reusable, runnable code a student can build from directly.

2009–2018 — Compiler verification & testing (5 sources)

Shows two separate roads to trusting a compiler's output: a full formal proof (Leroy's CompCert) or testing without one, via metamorphic and equivalence-based methods (Tao et al.; Le, Afshari & Su; Segura et al.; Chen et al.). None of this testing literature targets deliberately hidden backdoors — only ordinary correctness bugs.

2018–2020 — Self-replication, built for real (3 sources)

The only prior work that actually constructs a self-replicating attack outside Thompson's own toy example (Skrimstad, targeting the Go compiler), alongside adjacent work on reproducible builds and self-hosting trust bootstrapping (Linderud; Somlo). Confirms the attack generalizes across toolchains, but each effort stops at one implementation with no original detector attached.

2020–2023 — Supply-chain security at scale (6 sources)

Documents that build-time compromise is a live, large-scale category of real attack (the SolarWinds/SUNSPOT incident) and that reproducible builds are the industry's main proposed mitigation (Lamb & Zacchiroli, plus three SoK-style surveys). Operates at an infrastructure scale no single small project can fully instrument or inspect end to end.

2024–2026 — The pattern keeps generalizing (6 sources)

A real backdoor reaches production Linux infrastructure (the xz-utils incident, CVE-2024-3094); DDC-style thinking moves into CI/CD pipelines and binary-equivalence checking; and two 2025–26 papers extend the same trusting-trust logic to AI coding agents and to ordinary binary utilities (GNU strip), showing the mechanism keeps reappearing in new guises faster than verification techniques keep pace.

> **The throughline.** Across all 24 sources, the attack keeps generalizing and defenses keep being proposed — but nothing ties a from-scratch implementation of both to an original, formally-argued detector that is small enough to be fully inspected by one person. That specific gap is what this project fills.

Part V — The Novelty Argument

Be ready to explain not just the final number but how it was arrived at — faculty are far more likely to respect a score that was argued up honestly than one asserted from the start.

How the score actually evolved

First honest assessment: 5 / 10

Before any extensions, with only Phases 1–5: a working implementation of two well-established ideas (Thompson 1984, Wheeler 2005), fully verified but conceptually not new. Conceptual novelty alone would score 1–2; the completeness and safety-scoped redesign pulled it to a defensible 5.

After the extensions and arms race: 7, then 8 / 10

Adding independent detectors, a genuine evasion-and-counter-evasion contest, and an ensemble that had two real bugs found and fixed through adversarial self-testing moved this from “ran known techniques” to “attacker and defender actually adapt to each other, with evidence.”

After the original synthesis and formal proof: 9 / 10

The metamorphic method is a genuine synthesis verified against the literature as not previously combined this way, and the formal proof's prediction was confirmed by re-running the live tool — not merely asserted. This is the evidence chain that supports 9 without overclaiming a 10.

Why 9, and not 10

A 10 would mean no reasonable expert could point to anything derivative in the whole project. That is not true here: the base attack and defense are still Thompson's and Wheeler's ideas, and metamorphic testing as a general technique is also not this project's invention. What is original is the specific combination, the formal characterization of what it can and cannot detect, and the fact that theory and implementation were cross-checked against each other rather than asserted separately. If your faculty pushes for a 10, the honest answer is: the ideas being combined are not new; the combination, the proof, and the live verification of that proof's prediction are.

The six-step evidence chain to recite

1. A literature search confirmed the gap first — not assumed, checked.
2. An original synthesis: metamorphic testing applied to trusting-trust detection.
3. Fully implemented and empirically verified — every result was actually run.
4. A formal soundness proof, grounded in this project's own real source code.
5. The proof's prediction was confirmed by re-running the live tool afterward.
6. What the method provably cannot detect is stated as precisely as what it can.

Part VI — Advantages and Disadvantages

Advantages

- Every payload across every phase is safety-scoped — always inert, never a functional credential bypass.
- Every claim is independently verified: hashes, generation counts, pass/fail logs — not narrated.
- Three real bugs found and fixed during development (Phase 4's substitution order, and the ensemble's two bugs) are disclosed openly, which strengthens credibility rather than weakening it.
- The metamorphic method needs zero trusted reference compilers to operate — a genuinely new capability within this project.
- A formal proof backs the strongest claim, checkable line by line against the real source code, not just asserted in prose.

Disadvantages

- The base attack and defense are not novel — Thompson (1984) and Wheeler (2005) remain the source ideas.
- The metamorphic method is sound but incomplete: it is provably unable to detect Class B (AST-internal, name-keyed) logic.
- TinyLang is a toy language; the compositional-codegen assumption behind MR2 may not hold for a real, optimizing compiler.
- The arms race models only two rounds — a more adaptive attacker (e.g. one that detects randomization itself) is not modeled.
- Detector timing at small input sizes is dominated by process-startup overhead, not algorithmic scaling — stated honestly rather than presented as a clean benchmark.

Part VII — Anticipated Questions From Faculty

Q: Is any of this your own original idea, or is it all reproduced from Thompson and Wheeler?

A: The base attack and defense are not original — full credit belongs to Thompson (1984) and Wheeler (2005/2009). What is original is Phase 6: the specific evasion-and-counter-evasion contest I designed and ran, the ensemble detector (including two real bugs I found and fixed), and specifically the metamorphic testing method in Part 3, which a literature search did not find combined with trusting-trust detection before, plus the formal proof behind it in Part 4.

Q: Why should I believe your novelty score instead of treating it as self-serving?

A: Because it's not asserted — it's argued with a six-step evidence chain (Part V), and every step has a corresponding artifact: a documented literature search, real code, a real proof, and a real terminal run confirming the proof's prediction. I'd rather you push on any one of those six steps than take the number on faith.

Q: What exactly can your detection method NOT catch?

A: Anything where the backdoor's trigger condition is itself a legitimate part of the program's syntax tree — for example, checking a function's own name, which is what the plain, multitarget, and obfuscated variants in this project do. This is not a gap I found by accident; the formal proof in Part 4 predicts it exactly, and I verified the prediction by running the tool against those variants.

Q: If the method has a proven blind spot, why do you still call it useful?

A: Because it's a different CLASS of evidence from everything else I built — it needs no trusted reference compiler at all, which DDC and the fuzzer both require. The formal proof shows precisely why combining it with a reference-based method (DDC) covers both classes of dishonesty, which neither one covers alone. That argument, not the method in isolation, is the actual contribution.

Q: Why does the payload only print a message instead of doing something real, like bypassing a password?

A: Every structural property Thompson described — detection, injection, self-replication, evasion, and detection of the evasion — is fully demonstrated whether the payload is a real credential bypass or an inert, clearly-labeled marker. Making it inert keeps every lesson of the project intact while keeping the artifact safe to build, run, and hand in.

Q: How do you know your compiler's output is actually deterministic, rather than just assuming it?

A: I didn't assume it — I tested it. The same TinyLang file was compiled three separate times and all three outputs were hashed with SHA-256; all three hashes matched exactly. That property is what later lets Phase 5's DDC harness treat any hash mismatch as meaningful rather than noise.

Q: What was the hardest technical problem you actually had to solve yourself?

A: Getting the self-replicating compiler's source to correctly embed a copy of itself (Phase 4) without corrupting its own syntax. I made a real mistake here — substituting two placeholder tokens in the wrong order, which caused a nested, spurious duplicate that broke the regenerated file. I only found it by actually compiling the output and watching it fail, then fixed it by reordering the substitutions.

Q: Does your project generalize beyond this one toy language and this one function?

A: Partially, and I say so honestly. The multi-target injection (Phase 6, Part 1) shows the attack generalizes to any function matching a keyword pattern, not just one hardcoded name. But TinyLang itself is a toy language, and the compositional-codegen assumption behind MR2 specifically may not hold for a real compiler doing legitimate cross-function optimizations like inlining — that's listed directly as a disadvantage in Part VI, not hidden.

Part VIII — Glossary of Key Terms

AST (Abstract Syntax Tree)	A tree representation of a program's structure, stripped of surface syntax like parentheses and semicolons.
DFA (Deterministic Finite Automaton)	A state machine used by the lexer: one state plus one input character always leads to exactly one next state.
Quine	A program whose output, when run, is an exact copy of its own source code.
Diverse Double Compiling (DDC)	Wheeler's technique: build a compiler two independent ways, then use both results to compile the source again and compare outputs.
Trusting Trust attack	Thompson's 1984 demonstration that a compiler can inject hidden logic into a target program and into itself, with no trace left in any source file.
Metamorphic testing	A software-testing technique that checks whether a program behaves consistently across transformations known in advance to be meaning-preserving.
Metamorphic Relation (MR)	A specific transformation, plus the relationship its output must satisfy for a program to be considered correct/consistent.
Grammar-null transformation	A transformation proven, from the language's own grammar, to leave the resulting syntax tree unchanged.
AST-faithful / honest compiler	A compiler whose output is a pure function of the program's syntax tree alone — nothing else influences it.
Class A dishonesty	A backdoor that depends on something outside the AST — raw bytes, file identity, timing. Detectable by metamorphic testing alone.
Class B dishonesty	A backdoor that is technically a pure function of the AST, but includes illegitimate logic (e.g. keyed on a function's own name). Provably invisible to metamorphic testing alone.
Ensemble detector	A combined verdict built from several independent detectors, flagging a compiler as infected if enough of them agree.
Fixed point (in self-replication)	The point at which successive generations of a self-replicating compiler produce identical output, meaning the infection will propagate unchanged forever.

References

All 24 sources surveyed for the literature review, in chronological order. Full venue details are abbreviated for space; ask if you need a fully expanded citation for any specific entry before your presentation.

1. Karger, P. A. & Schell, R. R. (1974). Multics Security Evaluation: Vulnerability Analysis. ESD-TR-74-193, USAF.

2. Thompson, K. (1984). Reflections on Trusting Trust. Communications of the ACM, 27(8), 761–763.

3. Wheeler, D. A. (2005). Countering Trusting Trust through Diverse Double-Compiling. ACSAC.

4. Wheeler, D. A. (2009). Fully Countering Trusting Trust through Diverse Double-Compiling. PhD dissertation, George Mason University.

5. Leroy, X. (2009). Formal Verification of a Realistic Compiler. Communications of the ACM, 52(7).

6. Tao, Q., Wu, W., Zhao, C., & Shen, W. (2010). An Automatic Testing Approach for Compiler Based on Metamorphic Testing Technique. APSEC.

7. Le, V., Afshari, M., & Su, Z. (2014). Compiler Validation via Equivalence Modulo Inputs. PLDI.

8. Segura, S., Fraser, G., Sanchez, A. B., & Ruiz-Cortés, A. (2016). A Survey on Metamorphic Testing. IEEE Transactions on Software Engineering.

9. Chen, T. Y. et al. (2018). Metamorphic Testing: A Review of Challenges and Opportunities. ACM Computing Surveys.

10. Skrimstad, Y. (2018). Improving Trust in Software through Diverse Double Compiling and Reproducible Builds. MSc thesis.

11. Linderud, M. (2019). Reproducible Builds: Break a Log, Good Things Come in Trees. MSc thesis, University of Bergen.

12. Ohm, M., Plate, H., Sykosch, A., & Meier, M. (2020). Backstabber's Knife Collection: A Review of Open Source Software Supply Chain Attacks. DIMVA.

13. Somlo, G. (2020). Toward a Trustable, Self-Hosting Computer System. IEEE Security and Privacy Workshops.

14. CrowdStrike Intelligence (2021). SUNSPOT Implant Analysis (SolarWinds supply-chain compromise).

15. Lamb, C. & Zacchiroli, S. (2022). Reproducible Builds: Increasing the Integrity of Software Supply Chains. IEEE Software.

16. Okafor, C. et al. (2022). SoK: Analysis of Software Supply Chain Security by Establishing Secure Design Properties. SCORED.

17. Fourné, M., Wermke, D., Enck, W., Fahl, S., & Acar, Y. (2023). It's Like Flossing Your Teeth: On Reproducible Builds. IEEE S&P.

18. Ladisa, P., Plate, H., Martinez, M., & Barais, O. (2023). SoK: Taxonomy of Attacks on Open-Source Software Supply Chains. IEEE S&P.

19. CVE-2024-3094 (2024). xz-utils backdoor, public disclosure and subsequent analyses.

20. Anonymous (2024). Levels of Binary Equivalence for Comparison of Binaries from Alternative Builds. arXiv:2410.08427.

21. KTH Royal Institute of Technology (2024–25). Diverse Double Compiling in a Continuous Integration and Continuous Delivery Pipeline. MSc thesis.

22. Anonymous (2025). Beyond Trusting Trust: Multi-Model Validation for Robust Code Generation. arXiv:2502.16279.

23. Malka, J. et al. (2026). Trusting-Trust Attack against an Entire Linux Distribution through Binary Manipulation. arXiv:2607.24888.

24. Anonymous (2026). Reflections on Trusting Trust, Revisited: Contaminating Self-Modifying AI Coding Agents with Poisoned Benchmarks. arXiv:2609.17817.